Cod. Corso A757- 12-2024-0
ID Attività 2584660

Insegnamento: Zero-Knowledge Proofs (ZKP)

Prof. Silvio Meneguzzo

MODULE

Zero-Knowledge Proofs (ZKP)

Proving a statement is true while revealing nothing else: commitments, Sigma protocols, SNARKs & STARKs — the math, an on-chain Remix lab, and what powers privacy and zk-rollups

Blockchain and Distributed Ledger Technologies

Course module — Blockchain & DLT · CYBER 2026

COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

Outline

I · Foundations

The three properties, statement vs witness, interactive → non-interactive.

II · Commitments & Sigma

Hiding / binding commitments and the Schnorr proof of knowledge.

III · Lab

Verify a Schnorr proof on-chain in Remix, using the bn128 precompiles.

IV · SNARKs & STARKs

General-purpose succinct proofs for arbitrary computation.

V · Applications

Private payments, zk-rollups for scaling, identity and proof of computation.

VI · References

Foundational papers, toolchains and further reading.

PART I

Foundations

What a zero-knowledge proof is — and the three properties that define it

COESIONE ITALIA 21-27 PIEMONTE
Cofinanziato dall'Unione europea
REGIONE PIEMONTE

What is a zero-knowledge proof?

A protocol where a PROVER convinces a VERIFIER that a statement is true, while revealing nothing beyond its truth. Three properties define it.

Completeness

If the statement is true and the prover is honest, the verifier is always convinced.

Soundness

If the statement is false, no cheating prover can convince the verifier (except with negligible probability).

Zero-knowledge

The verifier learns NOTHING except that the statement is true — the transcript could have been faked without the secret.

Why on a blockchain

Verify a fact (a payment is valid, you are eligible) without publishing the private data behind it, on a public chain.

COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

Statement vs witness

The statement (public)

What you claim is true — e.g. 'this public key P corresponds to a private key I know', or 'I am over 18'.

The witness (secret)

The private data that makes it true — the private key x, your date of birth. The proof shows you HAVE it without showing it.

A ZKP is a 'proof of knowledge': it demonstrates the prover knows a witness for the statement — not merely that one exists.

Key idea: publish the statement, keep the witness; the proof convinces without leaking.

COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

Intuition — the Ali Baba cave

The setup

A ring-shaped cave has a locked door joining two paths, A and B. Peggy claims to know the secret word that opens it; Victor waits outside.

The proof

Peggy enters one path; Victor then shouts which path she must EXIT by. If she always exits correctly over many rounds, she must know the word — yet Victor never hears it.

One round: a lucky guesser succeeds 1/2 the time. After n rounds, cheating succeeds with probability $1/2^n$ → soundness. Victor learns nothing but the fact → zero-knowledge.

Quisquater & Guillou, 'How to Explain Zero-Knowledge Protocols to Your Children' (1990).

COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

Interactive → non-interactive

Classic ZKPs are a conversation. To put a proof on-chain we must remove the back-and-forth.

Interactive

Prover and verifier exchange messages; the verifier sends a fresh RANDOM challenge the prover cannot predict.

The Fiat-Shamir transform

Replace the verifier's random challenge with a HASH of the transcript so far. The prover cannot cheat because it cannot control the hash.

Non-interactive (NIZK)

The result is a single, self-contained proof anyone can verify later — exactly what a smart contract needs.

Random-oracle caveat

Fiat-Shamir is proven secure treating the hash as an ideal 'random oracle' — a standard, widely-accepted assumption.

COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

PART II

Commitments & Sigma protocols

The building blocks — and the Schnorr proof you will verify on-chain

COESIONE ITALIA 21-27 PIEMONTE
Cofinanziato dall'Unione europea
REGIONE PIEMONTE

Commitments — the core primitive

A commitment locks in a value now and reveals it later. You already used one in the commit-reveal lab (lab03).

Hiding

The commitment reveals nothing about the value before you open it.

Binding

You cannot open the commitment to a different value later — you are bound to what you locked.

Hash commitment

keccak256(value ‖ salt): hiding from pre-image resistance, binding from collision resistance. Simple and on-chain-cheap.

Pedersen commitment

value·G + salt·H on an elliptic curve: additively homomorphic and perfectly hiding — the basis of many ZK systems.

From commit-reveal to zero-knowledge

Commit-reveal (lab03)

Hides a move until BOTH players open it. But revealing eventually SHOWS the value — it is delayed disclosure, not zero-knowledge.

Zero-knowledge

Proves a property of the hidden value (e.g. 'I know its pre-image', 'it is in this set') WITHOUT ever revealing the value itself.

A Sigma protocol turns a commitment into a proof of knowledge: commit to randomness, answer a challenge, and the verifier checks an equation — learning nothing about the secret.

Key idea: commit-reveal hides then shows; ZK proves without ever showing.

The Schnorr proof of knowledge

Prove you KNOW the private key x behind a public key P = x·G, revealing nothing about x. This is the 'hello world' of ZK (and the lab).

Sigma protocol + Fiat-Shamir

```
Statement (public):  P = x·G          Witness (secret):  x

Prover:   pick random k    R = k·G                (commitment)
          c = H(P, R) mod n                      (challenge)
          s = k + c·x  (mod n)                   (response)
Proof = (R, s)

Verifier accepts  <=>  s·G == R + c·P
  because  s·G = (k + c·x)·G = k·G + c·(x·G) = R + c·P
```

G is the curve generator; n is the group order. R is random and s is masked by k, so x never leaks.

Why Schnorr is zero-knowledge

The three properties, made concrete for this protocol.

Complete

An honest prover who knows x always satisfies $s \cdot G = R + c \cdot P$.

Sound

Two valid responses to different challenges would let an extractor solve for x — so forging without x means breaking discrete log.

Zero-knowledge

The transcript (R, c, s) is simulatable without x, so it leaks nothing about the secret.

Nonce hygiene

k must be fresh and secret every time. Reusing k across two proofs leaks x (the flaw that broke the Sony PS3 keys).

PART III

Lab — verify a proof on-chain

A real Schnorr ZKP, verified in Remix with the bn128 precompiles

COESIONE ITALIA 21-27 PIEMONTE
Cofinanziato dall'Unione europea
REGIONE PIEMONTE

LAB — DO IT NOW

Verify a Schnorr proof on-chain

Open the folder lab-zkp/ in Remix (VM). Complete SchnorrVerifier.verify in 01-SchnorrZKP_start.sol (the check s·G == R + c·P), then use the prover harness to produce a proof and verify it — and watch a tampered proof return false. The next slides explain the solution.

COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

Lab solution — verifying on-chain

The verifier recomputes the challenge and checks the curve equation using Ethereum's elliptic-curve precompiles.

SchnorrVerifier.sol

```
function verify(uint Px, uint Py, uint Rx, uint Ry, uint s)
        public view returns (bool) {
  if (s >= N) return false;
  uint c = uint(keccak256(abi.encodePacked(Px,Py,Rx,Ry))) % N;
  (uint lx, uint ly) = ecMul(GX, GY, s);      // s·G     (0x07)
  (uint px, uint py) = ecMul(Px, Py, c);      // c·P     (0x07)
  (uint rx, uint ry) = ecAdd(Rx, Ry, px, py); // R + c·P (0x06)
  return (lx == rx && ly == ry);
}
```

ecMul / ecAdd call the bn128 precompiles at 0x07 and 0x06 — the same ones a real Groth16 verifier uses.

COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

What the lab demonstrates

A genuine ZKP, end-to-end, entirely inside the Remix VM.

On-chain verification

The verifier only ever sees P, R and s — never the secret x — yet is convinced the prover knows it.

Real curve math

Scalar multiplication and point addition run on the bn128 precompiles (0x07, 0x06) — no library, no network.

Soundness, felt

Change one digit of s and verify returns FALSE: a forged response cannot satisfy the equation.

Nonce reuse leaks x

The lab notes: reuse the nonce k across two proofs and the secret can be recovered — never reuse randomness.

COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

PART IV

SNARKs & STARKs

From proving one fact to proving an entire computation, succinctly

From one statement to any computation

Schnorr proves one specific fact. General-purpose ZK proves 'I correctly ran THIS program' for an arbitrary program.

Circuits

The computation is expressed as an arithmetic circuit — additions and multiplications over a finite field.

Arithmetization

The circuit becomes a system of polynomial constraints (e.g. R1CS / PLONKish) that hold iff the computation was correct.

Polynomial commitments

The prover commits to polynomials (e.g. KZG) and proves the constraints hold at random points — the engine of modern SNARKs.

Succinct

The proof is tiny and fast to verify, even if the computation was huge — the 'S' in SNARK.

COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

SNARK vs STARK

zk-SNARK

Succinct Non-interactive ARgument of Knowledge. Very small proofs, cheap on-chain verification; many schemes need a trusted setup and use pairing-based crypto.

zk-STARK

Scalable Transparent ARgument of Knowledge. No trusted setup and post-quantum (hash-based), but proofs are larger and cost more to verify.

Both are succinct relative to the computation and both hide the witness. The choice trades setup assumptions and proof size against verification cost.

Key idea: SNARK = smallest proofs (often a setup); STARK = no setup, quantum-safe, bigger proofs.

A map of proving schemes

The trade-off is mostly about the SETUP and the proof size.

Groth16

Pairing-based SNARK with the smallest proofs (a few group elements), but a trusted setup PER circuit.

PLONK

Universal, updatable trusted setup — one ceremony serves many circuits. Basis of much of today's tooling.

STARKs

Transparent (no setup), hash-based and post-quantum; used by Starknet. Larger proofs.

Newer schemes

Halo2, Nova (folding) and others push proving cost and recursion further — a fast-moving research area.

Verifying a SNARK on Ethereum

The proof is generated OFF-chain; only a small, cheap verification runs on-chain.

circom + snarkjs

Write a circuit, compile it, run a setup, and export a Solidity verifier contract automatically.

The pairing precompile

A Groth16 verifier checks a pairing equation via the bn128 precompile at 0x08 (EIP-197) — plus 0x06 / 0x07 you used in the lab.

Cheap to verify

Verification is a fixed, small gas cost regardless of the proven computation's size — this is what makes zk-rollups viable.

Expensive to prove

Proof generation is heavy and done off-chain by a specialized prover; the chain only pays to verify.

Polynomial commitments — KZG (exam topic)

What KZG is

A polynomial commitment: commit to a polynomial with one small elliptic-curve element, then prove its value at any point with a constant-size opening. Verified with a pairing.

Where it is used

The commitment engine inside PLONK-family SNARKs — and, on Ethereum, inside EIP-4844 'blob' transactions (proto-danksharding) that cut rollup data costs.

KZG needs a universal trusted setup (the 'powers of tau' ceremony); Ethereum ran a large public ceremony for EIP-4844.

Key idea: polynomial commitments let a verifier check a huge polynomial by testing it at a few random points.

Real case — Zcash's trusted-setup flaw (2018)

Why the trusted setup matters: a subtle bug in Zcash's original setup could have allowed INFINITE counterfeiting — silently.

What Zcash is

Zcash is a privacy coin using zk-SNARKs to shield transactions. Its 2016 launch relied on a per-circuit trusted setup (the “toxic waste” must be destroyed).

The flaw

In 2018 cryptographer Ariel Gabizon found the setup (from the BCTV14 paper) produced extra “bypass elements” — anyone with the setup transcript could forge valid-looking proofs.

The impact

The bug allowed counterfeiting UNLIMITED shielded ZEC, undetectably. It did NOT affect user privacy, and no evidence was found that it was ever exploited.

Lesson

A SNARK is only as sound as its setup. It was fixed by the Sapling upgrade (Oct 2018) — exactly why modern setups use large, transparent “powers of tau” ceremonies (e.g. EIP-4844).

PART V

Applications

Where zero-knowledge is already reshaping blockchains

COESIONE ITALIA 21-27 PIEMONTE
Cofinanziato dall'Unione europea
REGIONE PIEMONTE

What ZKPs unlock

Two families of use: PRIVACY (hide the data) and SCALING (compress the work).

Private payments

Prove a transfer is valid without revealing sender, receiver or amount (Tornado Cash, Zcash, Aztec).

Scaling — zk-rollups

Prove thousands of transactions were executed correctly, then post one succinct validity proof to L1.

Private identity

Prove 'over 18' or 'a citizen' from a credential without revealing the document (Semaphore, zk-passport work).

Proof of computation

Prove an off-chain computation (even an ML inference — zkML) ran correctly, verified cheaply on-chain.

COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

zk-rollups — validity proofs for scaling

An L2 executes transactions off-chain and proves correctness to Ethereum with a ZK proof.

How it works

Batch many L2 transactions, generate one validity proof, post it + compressed data to L1, which verifies it.

vs optimistic rollups

No 7-day fraud-proof window: a validity proof gives near-instant, cryptographic finality on L1.

Live networks

zkSync Era, Starknet (STARK), Scroll, Linea, Polygon zkEVM — all verify proofs against Ethereum.

Why it scales

L1 verifies a small proof instead of re-executing every transaction — throughput without re-doing the work.

COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

The privacy pattern — commitments + nullifiers

Deposit

You deposit funds behind a COMMITMENT added to an on-chain Merkle tree — the link to your identity is hidden.

Withdraw

You prove in zero-knowledge that you know a leaf of the tree, and publish a NULLIFIER (derived from the secret) so the note cannot be spent twice.

The proof of MEMBERSHIP is the 'proof of knowledge' part (like the lab's Schnorr); a full SNARK circuit proves it without revealing WHICH leaf.

Key idea: a nullifier prevents double-spends while the ZK proof keeps the spender anonymous.

COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

Key takeaways

- A ZKP proves a statement true while revealing nothing else — completeness, soundness, zero-knowledge.
- Statement is public, witness is secret — a ZKP is a proof of KNOWLEDGE of the witness.
- Fiat-Shamir makes proofs non-interactive — a single proof a smart contract can verify.
- Schnorr is the 'hello world' — the lab verifies $s \cdot G = R + c \cdot P$ on the bn128 precompiles.
- SNARKs / STARKs generalize this to any computation — small proofs, cheap on-chain verification.
- This powers privacy and zk-rollups — validity proofs, private payments, private identity.

COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

PART VI

References

Foundational papers, toolchains and further reading

References

[1] Goldwasser, Micali, Rackoff — 'The Knowledge Complexity of Interactive Proof-Systems' (1985).

[2] Quisquater & Guillou — 'How to Explain Zero-Knowledge Protocols to Your Children' (1990).

[3] Fiat & Shamir — 'How to Prove Yourself' (1986); Schnorr — 'Efficient Identification and Signatures' (1989).

[4] Groth16 (2016); PLONK (2019); STARKs — Ben-Sasson et al. (2018). eprint.iacr.org

[5] KZG polynomial commitments (2010); Ethereum EIP-4844 (proto-danksharding). eips.ethereum.org/EIPS/eip-4844

[6] circom & snarkjs — write circuits and export Solidity verifiers. docs.circom.io

[7] bn128 precompiles — EIP-196 (ECADD/ECMUL) and EIP-197 (pairing). eips.ethereum.org

[8] Vitalik Buterin — 'zk-SNARKs under the hood' series; ethereum.org/zero-knowledge-proofs.

COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

Questions?

Zero-knowledge lets a public chain verify a fact without seeing the data behind it — the foundation of on-chain privacy and of scaling by proof, not by re-execution.

COESIONE ITALIA 21-27 PIEMONTE
Cofinanziato dall'Unione europea
REGIONE PIEMONTE

Grazie per l’attenzione